Guide d'installation et de lancement de Bzz

Ce guide explique comment préparer un **Mac (Apple Silicon)** ou un **PC Windows avec une carte NVIDIA**, puis lancer le projet : la simulation dans **sa propre fenêtre Godot**, le tableau de bord dans le navigateur, et les décisions des abeilles par le modèle OpenJev.

Il suffit de faire la préparation (sections 2 ou 3) **une seule fois** par machine. Pour chaque séance de travail, il suffit ensuite de suivre la section 4.

1. Ce qui tourne, et où



Composant	Où il tourne	Pourquoi
Godot (simulation + fenêtre d'affichage)	directement sur la machine	Docker ne peut pas ouvrir de fenêtre sur le bureau
Backend Python + tableau de bord	Docker (Compose)	rien d'autre à installer ; identique sur toutes les machines
Service de décision OpenJev	directement sur la machine	Docker sur Mac n'a pas accès au GPU

La **fenêtre Godot** montre le monde en direct, mais on n'y agit pas. Toutes les actions (ajouts, retraits, pause, vitesse) se font dans le **tableau de bord**.

Ce qu'il faut installer :

Outil	Mac	Windows	Rôle
Git	oui	oui	récupérer le projet
Docker Desktop	oui	oui	lancer le backend et le tableau de bord
Python 3.10 ou plus	oui	oui	faire tourner le service de décision
Pilote NVIDIA récent	—	oui	utiliser la carte graphique (CUDA)
Godot 4.7.2	oui	oui	la simulation et sa fenêtre (même version que l'image Docker)

Espace disque : environ 10 Go au total (images Docker, environnement Python du service, et 3,2 Go de poids du modèle).

Mémoire : le modèle utilise environ 7,7 Go sur Mac (mémoire unifiée), environ 4 Go de VRAM sur NVIDIA. Prévoir 16 Go de mémoire au total sur Mac.

2. Préparer un Mac (Apple Silicon)

Les commandes se tapent dans l'application **Terminal**.

2.1 Homebrew

Homebrew installe les autres outils. S'il n'est pas déjà présent (`brew --version` affiche une erreur), suivre l'instruction d'installation de https://brew.sh, puis fermer et rouvrir le Terminal.

2.2 Git, Python et Docker Desktop

```
brew install git python@3.12
brew install --cask docker-desktop
```

Ouvrir ensuite **Docker Desktop** depuis le dossier Applications, accepter les conditions et attendre que la baleine de la barre de menus indique « Docker Desktop is running ».

2.3 Godot

```
brew install --cask godot
godot --version          # doit commencer par 4.7.2
```

Si la version installée diffère, télécharger Godot 4.7.2 sur https://github.com/godotengine/godot/releases/tag/4.7.2-stable et indiquer son chemin dans `.env` (`GODOT_BIN=/Applications/Godot.app/Contents/MacOS/Godot`). Le script de lancement refuse une autre version.

2.4 Vérifier

```
git --version
python3 --version        # 3.10 ou plus
docker --version
docker compose version
```

3. Préparer un PC Windows avec une carte NVIDIA

Les commandes se tapent dans **PowerShell**, sauf indication contraire.

> Cette configuration n'a pas encore été testée sur une vraie machine. En cas d'écart, voir le §7 et signaler le problème.

3.1 Pilote NVIDIA

Installer le pilote le plus récent de la carte depuis https://www.nvidia.com/drivers (« Game Ready » ou « Studio »), puis redémarrer. Vérifier :

```
nvidia-smi
```

La commande doit afficher le nom de la carte (par exemple « 16376MiB » de mémoire) et une version CUDA.

3.2 WSL 2, nécessaire à Docker Desktop

Dans PowerShell **ouvert en administrateur** (clic droit → « Exécuter en tant qu'administrateur ») :

```
wsl --install
```

Redémarrer le PC à la fin.

3.3 Git, Python et Docker Desktop

```
winget install --id Git.Git -e
winget install --id Python.Python.3.12 -e
winget install --id Docker.DockerDesktop -e
```

- Fermer puis rouvrir PowerShell pour que les nouvelles commandes soient reconnues.
- Lancer **Docker Desktop** depuis le menu Démarrer, accepter les conditions, puis attendre l'état « Engine running ».
- Sans `winget`, télécharger les installateurs sur https://git-scm.com, https://www.python.org/downloads/ (cocher « Add python.exe to PATH » et le lanceur `py`) et https://www.docker.com/products/docker-desktop/.

3.4 Godot

```
winget install --id GodotEngine.GodotEngine -e
```

Sinon, télécharger Godot 4.7.2 (« Windows 64-bit ») sur https://github.com/godotengine/godot/releases/tag/4.7.2-stable et indiquer le chemin de l'exécutable dans `.env` (`GODOT_BIN=C:\chemin\vers\Godot_v4.7.2-stable_win64.exe`). Le script de lancement cherche Godot dans le `PATH` et dans les dossiers d'installation de winget, vérifie sa version et refuse une autre version que 4.7.2.

3.5 Vérifier

```
git --version
py -3 --version          # 3.10 ou plus
docker --version
docker compose version
nvidia-smi
```

4. Lancer le projet (Mac et Windows)

4.1 Récupérer le projet (une seule fois)

```
git clone https://github.com/amineyoussefi-cpu/Bzz.git
cd Bzz
```

Ensuite, `git pull` dans ce dossier récupère les dernières modifications.

4.2 Configuration (facultative)

Tous les réglages ont une valeur par défaut. Pour en changer, copier le modèle de configuration, puis modifier `.env` :

```
cp .env.example .env              # Mac
Copy-Item .env.example .env       # Windows (PowerShell)
```

Réglages les plus utiles :

Variable	Défaut	Effet
`GODOT_DISPLAY`	`window`	`window` : Godot dans sa fenêtre sur la machine ; `headless` : sans fenêtre (§5)
`DASHBOARD_PORT`	`8000`	port du tableau de bord
`WORLD_WIDTH`, `WORLD_HEIGHT`	1000 × 700	taille du monde
`OPENJEV_DEVICE`	`auto`	`cuda`, `mps` ou `cpu` pour forcer l'appareil du modèle
`DECISION_INTERVAL_TICKS`	`120`	fréquence des décisions, en ticks
`DAY_TICKS`	`3600`	durée d'une journée simulée en ticks (1 minute réelle à ×1)
`SWARM_CLUSTER_DAYS`	`3`	jours dont dispose un essaim en grappe
`FLIGHT_SPEED_FACTOR`	`0.05`	vitesse d'affichage des vols (plus grand = vols plus rapides)

La liste complète est commentée dans `.env.example`.

4.3 Étape 1 : démarrer le service de décision

Dans un **premier terminal**, à la racine du projet :

Mac

```
./decision/start.sh
```

Windows

```
powershell -ExecutionPolicy Bypass -File .\decision\start.ps1
```

- **Au premier lancement**, le script installe l'environnement Python du service (environ 1 Go), puis télécharge les poids du modèle (3,2 Go). Compter 5 à 15 minutes selon la connexion. Sur Windows, il installe la version de PyTorch adaptée à la carte : CUDA 12.6, ou 13.0 pour les RTX 50xx.
- **Les fois suivantes**, le service démarre en quelques secondes, sans Internet.
- Le service est prêt quand il affiche une ligne de ce type :

```
OpenJev ready: model=alanhuangya/OpenJev-1.7B revision=7418ae8b6e3b device=mps (requested auto) loaded in 2.5s
```

`device` doit valoir **`mps`** sur Mac et **`cuda`** sur Windows.

Laisser ce terminal ouvert : fermer le terminal arrête le service.

4.4 Étape 2 : ouvrir la fenêtre Godot

Dans un **deuxième terminal**, à la racine du projet :

```
./godot/start.sh                                                    # Mac
powershell -ExecutionPolicy Bypass -File .\godot\start.ps1          # Windows
```

- Une **fenêtre « Bzz — simulation »** s'ouvre. Tant que le monde est vide, elle invite à ajouter des entités depuis le tableau de bord.
- Dans la fenêtre : **molette** = zoom, **glisser** = déplacer la vue, **F** = tout voir, **clic** ou survol = détails d'une entité. Elle ne permet aucune modification du monde.
- **Fermer la fenêtre arrête la simulation** (le monde est perdu). Relancer le script : le tableau de bord se reconnecte tout seul.

4.5 Étape 3 : démarrer le tableau de bord

Dans un **troisième terminal** (ou le deuxième, après avoir lancé la fenêtre) :

```
docker compose up -d
```

- Le premier lancement construit l'image Docker (quelques minutes) ; les suivants prennent quelques secondes.
- L'ordre n'est pas strict : si le tableau de bord démarre avant la fenêtre Godot, il attend et se connecte dès qu'elle s'ouvre.

4.6 Étape 4 : utiliser le tableau de bord

Ouvrir **http://localhost:8000** dans le navigateur, à côté de la fenêtre Godot.

1. **Vérifier** l'en-tête : « Godot : fenêtre » en vert. La section « Décisions des abeilles » doit indiquer le service **disponible**, avec le modèle, la révision et l'appareil.
2. **Ajouter des habitats** : choisir un type, cliquer dans la vue simplifiée du monde (dans le navigateur) pour placer, puis « Ajouter l'habitat ». L'habitat apparaît aussitôt dans la fenêtre Godot.
3. **Ajouter des abeilles** : choisir l'espèce, le rôle et la personnalité (préréglage, curseurs, ou aléatoire avec une graine), placer, puis « Ajouter l'abeille ». Chaque espèce n'accepte que certains habitats ; une abeille sans habitat compatible est affichée « en attente ».

4. **Poser un essaim** d'abeilles mellifères (formulaire « Ajouter un essaim ») : population, personnalité collective, position de la branche. L'essaim attend en grappe et dispose de quelques jours simulés pour trouver une cavité d'arbre ou une ruche libre.
5. **Lancer** la simulation (bouton en haut). Le monde est construit à l'arrêt, et les abeilles ne décident que lorsque la simulation tourne. Le jour et l'heure simulés s'affichent en haut ; la vitesse ×4 ou ×8 accélère tout.
6. **Suivre** les décisions et les vols **dans la fenêtre Godot** : chaque abeille ou essaim choisit (ou refuse) un habitat, **s'y rend en vol** (trajet en pointillés), puis s'installe à l'arrivée. Un essaim qui ne trouve rien avant la fin de son délai reste « à découvert ».
7. **Mettre en pause**, retirer une entité (✕) ou « Vider le monde » à tout moment.

4.7 Arrêter

1. Fermer la **fenêtre Godot** : la simulation s'arrête, et le monde est perdu (pas de sauvegarde).
2. `docker compose down` : arrête le tableau de bord.
3. **Ctrl + C** dans le terminal du service de décision.

4.8 Relancer une autre fois (résumé)

```
cd Bzz
./decision/start.sh          # terminal 1   (Windows : powershell -ExecutionPolicy Bypass -File .\decision\s
./godot/start.sh             # terminal 2   (Windows : powershell -ExecutionPolicy Bypass -File .\godot\star
docker compose up -d         # terminal 3, puis ouvrir http://localhost:8000
```

5. Variantes

5.1 Godot sans fenêtre, dans Docker (mode headless)

Pour un serveur, des tests ou si l'on ne veut pas de fenêtre, ajouter dans `.env` :

```
COMPOSE_PROFILES=headless
GODOT_HOST_FROM_DOCKER=godot
```

Puis `docker compose up -d` lance aussi Godot, sans fenêtre, dans Docker. Le tableau de bord indique « Godot : headless ». Ne pas lancer `godot/start.sh` dans ce mode.

5.2 Sans Docker

Cette variante est utile pour développer.

Mac

```
python3 -m venv .venv
.venv/bin/pip install -r backend/requirements.txt
./decision/start.sh                                  # premier terminal
cd backend && ../.venv/bin/python -m bzz.server      # second terminal
```

Windows

```
py -3 -m venv .venv
.venv\Scripts\python -m pip install -r backend\requirements.txt
powershell -ExecutionPolicy Bypass -File .\decision\start.ps1      # premier terminal
# second terminal : indiquer le chemin de Godot (version "console"), puis lancer le backend
$env:GODOT_BIN = "C:\chemin\vers\Godot_v4.7.2-stable_win64_console.exe"
cd backend; ..\.venv\Scripts\python -m bzz.server
```

- Sans `GODOT_HOST`, le backend lance Godot lui-même, **sans fenêtre** par défaut. Avec `GODOT_DISPLAY=window` dans l'environnement, il l'ouvre en fenêtre.
- Pour utiliser la fenêtre lancée par `godot/start.sh`, définir plutôt `GODOT_HOST=127.0.0.1` avant de lancer le backend.
- Ouvrir ensuite http://localhost:8000.
- Sur Mac, si Godot n'est pas trouvé, définir aussi `GODOT_BIN` (par exemple `/Applications/Godot.app/Contents/MacOS/Godot`).

6. Vérifications utiles

Quoi	Mac	Windows
Service de décision	`curl http://127.0.0.1:8009/health`	`Invoke-RestMethod http://127.0.0.1:8009/health`
Backend	`curl http://localhost:8000/api/state`	`Invoke-RestMethod http://localhost:8000/api/state`
Journaux Docker	`docker compose logs -f backend`	idem
Tests (sans le modèle)	`docker compose run --rm -e GODOT_HOST= backend python -m unittest discover tests`	idem

7. Problèmes fréquents

Symptôme	Solution
Tableau de bord : « Service de décision OpenJev indisponible »	Vérifier que le terminal du service est ouvert et affiche « OpenJev ready ». Le tableau de bord se met à jour seul en quelques secondes.
Windows : le service tourne, mais le tableau de bord le dit indisponible	Docker n'atteint pas le service : ajouter `OPENJEV_HOST=0.0.0.0` dans `.env`, relancer le service, puis autoriser Python dans le pare-feu Windows si une fenêtre le demande.
Windows : `device=cpu` au lieu de `cuda`	Pilote NVIDIA absent ou trop ancien (§3.1), ou mauvaise version de PyTorch : supprimer le dossier `decision\.venv`, puis relancer `start.ps1`. Pour forcer une version, ajouter `TORCH_CUDA=cu130` (ou `cu126`) dans `.env`.
Windows : « l'exécution de scripts est désactivée »	Lancer le script avec `powershell -ExecutionPolicy Bypass -File .\decision\start.ps1`, comme indiqué.
`docker compose` : « Cannot connect to the Docker daemon »	Docker Desktop n'est pas démarré : l'ouvrir et attendre qu'il soit prêt.
En-tête : « Godot : non connecté »	La fenêtre Godot n'est pas ouverte ou a été fermée : relancer `godot/start.sh` (ou `start.ps1`). La reconnexion est automatique.
Windows : la fenêtre Godot est ouverte, mais « non connecté »	Le conteneur n'atteint pas Godot sur `127.0.0.1` : ajouter `GODOT_LISTEN=0.0.0.0` dans `.env`, relancer `start.ps1`, et autoriser Godot **uniquement sur les réseaux privés** dans le pare-feu (cette adresse rend Godot joignable depuis le réseau local).
« Godot ... trouvé, mais 4.7.2 est attendu »	Installer Godot 4.7.2 et indiquer son chemin avec `GODOT_BIN` dans `.env`.
Le port 8000 est déjà pris	Choisir un autre port dans `.env` (`DASHBOARD_PORT=8080`), puis `docker compose up -d`.
Les abeilles ne décident rien	La simulation est en pause, aucun habitat n'accepte leur espèce (état « en attente »), ou le service est indisponible.
« honey_bee lives in swarms »	Les abeilles mellifères s'ajoutent avec le formulaire « Ajouter un essaim ».
Premier lancement très long	C'est le téléchargement unique des poids (3,2 Go) et la construction des images Docker.
Mac : `Permission denied` sur un script	`chmod +x decision/start.sh godot/start.sh`, puis relancer.

8. Pour aller plus loin

- Service de décision OpenJev : paramètres détaillés, mesures et format des requêtes.
- API du projet : toutes les commandes et routes, avec des exemples.
- Rapport de comparaison : accord entre OpenJev et la logique de référence.
- PROJECT_STATUS.md : état du projet, limites et prochaines étapes.